Tabla de contenidos

1. Sincronización
2. Mecanismos de sincronización
    a. *Lock*
    b. *Locks* y *Context Managers*
    c. Señales entre *threads*: `Event`
    d. Otros métodos de coordinación entre *threads*
3. *Deadlocks*

Sincronización

En el *notebook* anterior, vimos qué eran los *threads*, cómo crearlos, y hablamos sobre herramientas que podrían ser útiles. Podemos hacer cosas **pseudo-paralelas**, ¿qué podría salir mal?

Hagamos dos *threads* que aumenten un mismo contador 10^6 veces cada uno. Lo que esperaríamos es que el valor final sea 2×10^6, ¿no es así?

```
import threading


class Contador:
    def __init__(self) -> None:
        self.valor = 0


def sumador(contador: int) -> None:
    for _ in range(10**6):   ### Propuesto: cambiar este valor a 10**2. ¿Se comporta igual?
        contador.valor += int(1) # Este int() redundante es solo para que el ejemplo funcione


contador = Contador()
t1 = threading.Thread(target=sumador, args=(contador,))
t2 = threading.Thread(target=sumador, args=(contador,))

t1.start()
t2.start()
t1.join()
t2.join()

print("Listo, nuestro contador vale", contador.valor)
```

```
Listo, nuestro contador vale 1194571
```

😱¿Qué pasó ahí? Como dice un viejo refrán:

> Un programador tenía un problema y decidió usar *threads*. Ahora tiene dos problemas.

En este ejemplo, intentamos modificar **al mismo tiempo (concurrentemente)** un mismo valor o recurso con dos *threads* distintos. Para entender por qué eso no siempre resulta como pensábamos, tomemos en cuenta que:

- Las operaciones de los *threads* pueden ser pausadas en cualquier momento para dar paso a otro *thread*.
- Es imposible saber cómo se entremezcla la ejecución de los *threads*. Es una decisión del sistema operativo. Por ejemplo, no podemos suponer que **siempre** se ejecutará una operación del *thread* 1, luego una del *thread* 2, una del *thread* 1, y así sucesivamente.

Descompongamos en un pseudocódigo –lo más granular posible– lo que hace la máquina en una iteración en cada `sumador`:

```
1. Leer el valor de contador.valor
2. Sumar 1 al valor anterior
3. Almacenar el valor obtenido en contador.valor
```

Ahora veamos un escenario posible:

```
- T1 lee 0 de contador.valor
- T1 suma 1 => 1
- T1 guarda 1 en contador.valor
- T1 lee 1 de contador.valor
- T1 se pausa
    - T2 lee 1 de contador.valor
    - T2 suma 1 => 2
    - T2 guarda 2 en contador.valor
    - T2 lee 2 de contador.valor
    - T2 suma 1 => 3
    - T2 guarda 3 en contador.valor
    - T2 se pausa
- T1 se reanuda
- T1 suma 1 => 2 (😩)
- T1 guarda 2 en contador.valor (😩😩😩)
- ...
```

Revisemos el caso mediante código:

```python
import time


def sumador(contador: int) -> None:
    nombre = threading.current_thread().name
    for _ in range(10):
        valor = contador.valor
        print(f"{nombre}: lee {valor}")
```

```
        nuevo_valor = valor + 1
        print(f"{nombre}: suma 1 => {nuevo_valor}")
        contador.valor = nuevo_valor
        print(f"{nombre}: guarda {nuevo_valor}")
        time.sleep(1)


contador = Contador()
t1 = threading.Thread(name="T1", target=sumador, args=(contador,))
t2 = threading.Thread(name="T2", target=sumador, args=(contador,))

t1.start()
t2.start()
t1.join()
t2.join()

print("Listo, nuestro contador vale", contador.valor)
```

```
T1: lee 0
T1: suma 1 => 1
T1: guarda 1
T2: lee 1
T2: suma 1 => 2
T2: guarda 2
T1: lee 2T2: lee 2
T2: suma 1 => 3
T2: guarda 3

T1: suma 1 => 3
T1: guarda 3
T2: lee 3T1: lee 3
T1: suma 1 => 4
T1: guarda 4

T2: suma 1 => 4
T2: guarda 4
T1: lee 4T2: lee 4
T2: suma 1 => 5
T2: guarda 5

T1: suma 1 => 5
T1: guarda 5
T1: lee 5T2: lee 5
T2: suma 1 => 6
T2: guarda 6

T1: suma 1 => 6
T1: guarda 6
T2: lee 6T1: lee 6
T1: suma 1 => 7
```

```
T1: guarda 7

T2: suma 1 => 7
T2: guarda 7
T1: lee 7T2: lee 7
T2: suma 1 => 8
T2: guarda 8

T1: suma 1 => 8
T1: guarda 8
T1: lee 8T2: lee 8
T2: suma 1 => 9
T2: guarda 9

T1: suma 1 => 9
T1: guarda 9
T2: lee 9T1: lee 9
T1: suma 1 => 10
T1: guarda 10

T2: suma 1 => 10
T2: guarda 10
T2: lee 10T1: lee 10
T1: suma 1 => 11
T1: guarda 11

T2: suma 1 => 11
T2: guarda 11
Listo, nuestro contador vale 11
```

La situación anterior nos enseña que **deberíamos asegurarnos** de que la operación de aumentar el contador (`contador.valor += 1`) sea **atómica**, es decir, que solo un *thread* la pueda ejecutar en todo momento. Dentro de un código puede haber una secuencia de instrucciones que no queremos que sean ejecutadas en paralelo. Esto quiere decir solo un *thread* debe encontrarse en ese sección de código. A esta secuencia de instrucciones le llamamos **sección crítica**.

La situación que vimos es una de muchas donde más de un *thread* debe compartir el acceso a determinados recursos, por ejemplo archivos, variables, etc. En estos escenarios, **solo uno** de los *threads* debe tener acceso al recurso y el resto debe quedar en espera para su uso. Cuando existe **concurrencia** múltiple a un recurso es posible controlar el acceso a este mediante mecanismos de **sincronización** entre los *threads*.

Mecanismos de sincronización

Ahora, veremos dos formas de coordinar o **sincronizar** nuestros *threads*.

Lock

El *lock* es una primitiva de sincronización de *threads*, provista por la clase `Lock` de la librería `threading`. Se utiliza para que sólo un *thread* pueda estar en una misma sección crítica a la vez. En otras palabras, el *lock* permite la sincronización para el acceso a los recursos compartidos entre dos o más *threads*.

Un *lock* puede estar **bloqueado** o **desbloqueado** (inicialmente está desbloqueado). Si un *thread* quiere entrar a una sección crítica, primero debe adquirir el *lock* mediante la operación `acquire()`. Una vez que el *thread* consigue adquirir el *lock*, lo deja bloqueado, haciendo que otros *threads* que quieran adquirir el mismo *lock* deben esperar. Cuando el *thread* quiera salir de la sección crítica, debe liberar el lock mediante `release()`, con lo que el *lock* queda desbloqueado, permitiendo que otro *thread* pueda adquirirlo.

Una analogía a los *locks* es tener una zona protegida por llave. El primer *thread* en acceder a dicha zona, se lleva la llave y nadie puede entrar (el lock está **bloqueado**) y solo cuando este salga de dicha zona, devuelve la llave (lock **desbloqueado**) permitiendo que otro *thread* acceda a dicha zona.

lock

Abajo, se ejemplifica el modo de usar un *lock*.

```
import threading

## El mismo lock para todos los threads
lock_global = threading.Lock()


def sumador_con_seccion_critica(contador: int, lock: threading.Lock) -
> None:
    for _ in range(10 ** 6):
        # Pedimos el lock antes de entrar a la sección crítica.
        lock.acquire()
        # --- Sección crítica ---.
        # Está garantizado que en estas líneas sólo habrá un thread a
la vez.
        contador.valor += int(1)
        # --- Fin de la sección crítica ---.
        # Liberamos el lock luego de salir de la sección crítica.
        lock.release()
```

Nuestro ejemplo inicial del contador ahora funcionará correctamente .

```
contador = Contador()
t1 = threading.Thread(target=sumador_con_seccion_critica,
args=(contador,lock_global))
t2 = threading.Thread(target=sumador_con_seccion_critica,
args=(contador,lock_global))
```

```
# Esto podría tomar algunos segundos...
t1.start()
t2.start()
t1.join()
t2.join()

print("Listo, nuestro contador vale", contador.valor)
```

```
Listo, nuestro contador vale 2000000
```

ATENCIÓN: el *lock* debe ser compartido

El *lock* que usan los *threads* **debe ser el mismo**. Si cada *thread* usa una instancia distinta de *lock*, todos podrán pasar el método `acquire()` y entrarán a la sección crítica sin problemas.

```
import threading

## Si cada thread usa su propio lock, no estamos protegiendonos
## de ningún acceso concurrente.


def sumador_con_seccion_critica_mala(contador: int) -> None:
    ## Cada thread crea su propio lock
    ## ♂♀ ESTO ESTÁ MAL ♂♀
    lock = threading.Lock()

    for _ in range(10 ** 6):
        # Pedimos el lock antes de entrar a la sección crítica.
        lock.acquire()
        # --- Sección crítica ---.
        contador.valor += int(1)
        # --- Fin de la sección crítica ---.
        # Liberamos el lock luego de salir de la sección crítica.
        lock.release()


contador = Contador()
t1 = threading.Thread(target=sumador_con_seccion_critica_mala,
args=(contador,))
t2 = threading.Thread(target=sumador_con_seccion_critica_mala,
args=(contador,))

t1.start()
t2.start()

t1.join()
t2.join()

print("Listo, nuestro contador vale", contador.valor)
```

```
Listo, nuestro contador vale 1572749
```

Locks y *Context Managers*

En Python los *locks* también pueden funcionar dentro de un *context manager* a través de la sentencia `with`. En este caso es el mismo `with` el que se encarga de llamar los métodos `acquire()` y `release()` de forma implícita. De esta forma el *lock* usado en la función `sumador_con_seccion_critica` quedaría como se indica a continuación:

```
import time


lock_global = threading.Lock()


def sumador(contador: int, lock: threading.Lock) -> None:
    nombre = threading.current_thread().name
    for _ in range(10):
        with lock:
            # --- Sección crítica ---.
            # Está garantizado que en estas líneas sólo habrá un
thread a la vez.
            valor = contador.valor
            print(f"{nombre}: lee {valor}")
            nuevo_valor = valor + 1
            print(f"{nombre}: suma 1 => {nuevo_valor}")
            contador.valor = nuevo_valor
            print(f"{nombre}: guarda {nuevo_valor}")
            time.sleep(1)
            # --- Fin de la sección crítica ---.


contador = Contador()
t1 = threading.Thread(name="T1", target=sumador,
args=(contador,lock_global))
t2 = threading.Thread(name="T2", target=sumador,
args=(contador,lock_global))

t1.start()
t2.start()

t1.join()
t2.join()
```

```
T1: lee 0
T1: suma 1 => 1
T1: guarda 1
T1: lee 1
T1: suma 1 => 2
T1: guarda 2
```

```
T2: lee 2
T2: suma 1 => 3
T2: guarda 3
T2: lee 3
T2: suma 1 => 4
T2: guarda 4
T2: lee 4
T2: suma 1 => 5
T2: guarda 5
T1: lee 5
T1: suma 1 => 6
T1: guarda 6
T1: lee 6
T1: suma 1 => 7
T1: guarda 7
T1: lee 7
T1: suma 1 => 8
T1: guarda 8
T1: lee 8
T1: suma 1 => 9
T1: guarda 9
T2: lee 9
T2: suma 1 => 10
T2: guarda 10
T2: lee 10
T2: suma 1 => 11
T2: guarda 11
T1: lee 11
T1: suma 1 => 12
T1: guarda 12
T1: lee 12
T1: suma 1 => 13
T1: guarda 13
T1: lee 13
T1: suma 1 => 14
T1: guarda 14
T2: lee 14
T2: suma 1 => 15
T2: guarda 15
T2: lee 15
T2: suma 1 => 16
T2: guarda 16
T1: lee 16
T1: suma 1 => 17
T1: guarda 17
T2: lee 17
T2: suma 1 => 18
T2: guarda 18
T2: lee 18
```

```
T2: suma 1 => 19
T2: guarda 19
T2: lee 19
T2: suma 1 => 20
T2: guarda 20
```

Señales entre *threads*: `Event`

Otro patrón común que aparece en coordinación entre *threads* es la comunicación de eventos. Esto se refiere a la idea de que un *thread* sea capaz de esperar a que cierto evento ocurra para continuar sus operaciones. Por ejemplo, un *thread* que debe esperar a que ciertos cómputos se hayan completado previamente para poder operar con esos resultados. El método `join` podría ayudar a este cometido. El *thread* que espera puede usar `join` del (de los) *thread(s)* que espera. Pero `join` solo funciona para esperar el **término** de un *thread*, pero

- ¿Qué ocurre si se desea esperar un evento que ocurre **durante** la vida de otro *thread*?
- ¿Podemos tener un comportamiento similar pero donde el segundo *thread* no tenga necesariamente que terminar para que el primer *thread* continúe?
- ¿Cómo podemos hacer que un *thread* espere a que otro le avise cuando continuar?

Para ello tenemos los objetos `Event`. Un *event* es uno de los mecanismos más simples de comunicación entre *threads*: un *thread* hace una señal, y otros *threads* esperan a que esa señal ocurra. Los `Event` tienen un *flag* interno, que toma el valor `True` cuando la señal está activa, y `False` cuando no.

- **Esperar**: Un *thread* puede esperar una señal llamando al método `wait()` del `Event`; con ello, el *thread* quedará en pausa hasta que otro *thread* active la señal correspondiente. En caso de que la señal ya haya estado activa antes de hacer `wait()`, el *thread* puede seguir inmediatamente sin esperar.

- **Avisar**: Para activar la señal, un *thread* debe llamar al método `set()`, que dejará el *flag* interno del objeto `Event` en `True`. Finalmente, un *thread* cualquiera puede **resetear** la señal llamando a `clear()` del objeto `Event`, dejando el *flag* en `False`.

- **Revisar**: Para revisar si una señal fue activada o no, se debe llamar al método `is_set()` de un *thread*. Este retornará `True` o `False` indicando el valor del *flag* del *thread* sin tener que pausar el thread como ocurre cuando se usa `wait()`.

En el siguiente ejemplo, los *thread 2 y 3* usan `E.wait()` para esperar que el evento `E` se active. Despues de un rato, el *thread 1* hace `E.set()` para que el evento `E` se active y así los *threads 2 y 3* pueden continuar con su ejecución.

Un ejemplo es cuando queremos reproducir un audio y un video de la forma más sincronizada posible. Supongamos que tenemos un *thread* encargado de leer el audio, y otro de leer el video. El *thread* encargado del audio debería esperar a que el *thread* encargado del video esté listo para empezar, y viceversa:

```python
# Ejemplo adaptado de http://zulko.github.io/blog/2013/09/19/a-basic-example-of-threads-synchronization-in-python/

import threading
import time


# Tenemos dos eventos o señales.
# Esta es para avisar que el video ya está listo para ser reproducido.
video_cargado = threading.Event()

# Esta es para avisar que el audio ya está listo para ser reproducido.
audio_cargado = threading.Event()

def hora_actual() -> None:
    return time.ctime().split(" ")[3]


def reproducir_video(nombre: str) -> None:
    print(f"[{hora_actual()}] Cargando video {nombre}. Se demorará 10 segundos\n")
    # Supongamos que se demora 10 segundos
    time.sleep(10)
    print(f"[{hora_actual()}] Video cargado. Esperando audio")
    # Avisamos que el video ya está cargado
    video_cargado.set()
    # Esperamos a que el audio ya se haya cargado
    audio_cargado.wait()
    # ¡Listo!
    print(f"[{hora_actual()}] Comenzando reproducción del video")


def reproducir_audio(nombre: str) -> None:
    print(f"[{hora_actual()}] Cargando audio {nombre}. Se demorará 3 segundos\n")
    # Supongamos que se demora 3 segundos
    time.sleep(3)
    print(f"[{hora_actual()}] Audio cargado. Esperando video")
    # Avisamos que el audio ya está cargado
    audio_cargado.set()
    # Esperamos a que el video ya se haya cargado
    video_cargado.wait()
    # ¡Listo!
    print(f"[{hora_actual()}] Comenzando reproducción del audio")


t1 = threading.Thread(target=reproducir_audio, args=("Your Name - Zen Zen Zense",))
t2 = threading.Thread(target=reproducir_video, args=("Your Name - Zen Zen Zense",))
```

```
t1.start()
t2.start()

t1.join()
t2.join()
```

```
[11:58:55] Cargando audio Your Name - Zen Zen Zense. Se demorará 3 segundos
[11:58:55] Cargando video Your Name - Zen Zen Zense. Se demorará 10 segundos


[11:58:58] Audio cargado. Esperando video
[11:59:05] Video cargado. Esperando audio
[11:59:05] Comenzando reproducción del video
[11:59:05] Comenzando reproducción del audio
```

En el ejemplo anterior, gracias a la coordinación de *threads* con *events*, conseguimos que el audio y el video se empiecen a reproducir **casi** simultáneamente. De otra manera, el audio habría empezado mucho antes, puesto que demoró solo 3 segundos en cargar, mientras que el video tardó 10 segundos.

El siguiente ejemplo hace uso se los métodos `set()`, `clear()`, `wait()`, `is_set()` de los *threads* en un mismo problema. Vamos a tener dos *threads*, los cuales simulan a un **trabajador** y a un **jefe** de un proyecto. El trabajador hará sus labores por una cantidad definida de semanas, y cada 4 semanas solicitará su sueldo, pausando sus labores hasta recibirlo. Por parte del jefe, este seguirá trabajando mientras el trabajador no haya terminado sus labores, además de periódicamente revisar si le han solicitado un sueldo o no. En caso de recibir la solicitud, este pagará al trabajador y esperará la confirmación de este, antes de seguir con sus labores. Una vez el trabajador terminé todas sus tareas, avisará y con eso tanto trabajador como jefe habrán finalizado el proyecto.

```
import time
import threading

# Tenemos cuatro eventos o señales
# Esta es para avisar que el trabajador solicitó su pago mensual
solicitar_pago = threading.Event()

# Esta es para avisar que el jefe pagó al trabajador
entregar_pago = threading.Event()

# Esta es para avisar que el trabajador recibió el pago
confirmar_recepcion = threading.Event()

# Esta le avisa al jefe que el trabajador terminó sus labores
finalizar_trabajo = threading.Event()
```

```python
# El trabajador trabaja cada 4 semanas y espera su pago
def trabajar_trabajador(semanas: int) -> None:
    print("[Trabajador] A chambear se ha dicho ♂")

    # Supongamos que se trabaja por semanas
    for semana in range(1, semanas + 1):
        print(f"[Trabajador] Trabajando la semana #{semana}")
        time.sleep(1)

        # Si la semana es múltiplo de 4, avisa que necesita pago
        # y deja de trabajar hasta que llegue el sueldo
        if not semana % 4:
            # Pide el pago e indica que no ha confirmado su recepción
            print("[Trabajador] ¡Jefe! Ya estamos a fin de mes □")
            confirmar_recepcion.clear()
            solicitar_pago.set()

            # Espera el pago
            entregar_pago.wait()
            print("[Trabajador] A seguir trabajando pues □")

            # Avisa la recepción del pago
            confirmar_recepcion.set()
            # Apaga la señal de solicitar pago para esperar 4 semanas
más
            solicitar_pago.clear()

    # Se acabó el proyecto a trabajar
    print("[Trabajador] Terminé todas mis labores.")
    finalizar_trabajo.set()


# El jefe deberá estar funcionando hasta que el proyecto termine
# Y además, revisar que se le pague al trabajador cuando lo pida
def trabajar_jefe() -> None:
    print("[Jefe] Tenemos que terminar este proyecto! ♀")

    finalizado = False
    while not finalizado:
        # Usa un is_set() para revisar si el trabajador ya terminó,
        # Pero no queremos detener el trabajo esperando el evento.
        # Es por esto que se usa is_set() en vez de wait() para
revisar.
        if finalizar_trabajo.is_set():
            finalizado = True
        else:
            print("[Jefe] Trabajando...")

            # El jefe en cada semana revisa si el trabajador ha
solicitado sueldo.
```

```
            if solicitar_pago.is_set():
                # Le paga al trabajador
                print("[Jefe] Aquí tiene su dinero estimado 💰")
                entregar_pago.set()

                # Ahora si, espera confirmación de recepción antes de
seguir
                confirmar_recepcion.wait()
                # Apaga la señal de entregar pago hasta la siguiente
solicitud
                entregar_pago.clear()
            else:
                print(f"[Jefe] Todavía no me piden el pago de este mes
💸")

            # Un sleep para esperar un poco antes de volver a revisar
            time.sleep(1)

    # Se acabó el proyecto a trabajar
    print("[Jefe] Podemos dar el proyecto por finalizado 🎉")


# El proyecto de trabajo durará 12 semanas para el trabajador
t_jefe = threading.Thread(target=trabajar_jefe)
t_trabajador = threading.Thread(target=trabajar_trabajador,
args=(12,))

t_jefe.start()
t_trabajador.start()

t_jefe.join()
t_trabajador.join()
```

```
[Jefe] Tenemos que terminar este proyecto! ♀
[Jefe] Trabajando...
[Jefe] Todavía no me piden el pago de este mes 💸
[Trabajador] A chambear se ha dicho ♂
[Trabajador] Trabajando la semana #1
[Jefe] Trabajando...
[Jefe] Todavía no me piden el pago de este mes 💸
[Trabajador] Trabajando la semana #2
[Jefe] Trabajando...
[Jefe] Todavía no me piden el pago de este mes 💸
[Trabajador] Trabajando la semana #3
[Jefe] Trabajando...
[Jefe] Todavía no me piden el pago de este mes 💸
[Trabajador] Trabajando la semana #4
[Jefe] Trabajando...
[Jefe] Todavía no me piden el pago de este mes 💸
[Trabajador] ¡Jefe! Ya estamos a fin de mes 💰
```

```
[Jefe] Trabajando...
[Jefe] Aquí tiene su dinero estimado 
[Trabajador] A seguir trabajando pues 
[Trabajador] Trabajando la semana #5
[Trabajador] Trabajando la semana #6[Jefe] Trabajando...
[Jefe] Todavía no me piden el pago de este mes 

[Jefe] Trabajando...[Trabajador] Trabajando la semana #7

[Jefe] Todavía no me piden el pago de este mes 
[Trabajador] Trabajando la semana #8[Jefe] Trabajando...
[Jefe] Todavía no me piden el pago de este mes 

[Jefe] Trabajando...[Trabajador] ¡Jefe! Ya estamos a fin de mes 

[Jefe] Aquí tiene su dinero estimado 
[Trabajador] A seguir trabajando pues 
[Trabajador] Trabajando la semana #9
[Trabajador] Trabajando la semana #10[Jefe] Trabajando...
[Jefe] Todavía no me piden el pago de este mes 

[Jefe] Trabajando...[Trabajador] Trabajando la semana #11

[Jefe] Todavía no me piden el pago de este mes 
[Trabajador] Trabajando la semana #12[Jefe] Trabajando...
[Jefe] Todavía no me piden el pago de este mes 

[Jefe] Trabajando...[Trabajador] ¡Jefe! Ya estamos a fin de mes 

[Jefe] Aquí tiene su dinero estimado 
[Trabajador] A seguir trabajando pues 
[Trabajador] Terminé todas mis labores.
[Jefe] Podemos dar el proyecto por finalizado 
```

Otros métodos de coordinación entre *threads*

En Python existen otras maneras de coordinar *threads*, que son adecuadas para otras situaciones. No las veremos en detalle en este curso. Si gustas, puedes verlas en la documentación, donde encontrarás información sobre objetos `RLock` (*lock* reentrante), objetos condición, objetos semáforo, objetos barrera y más.

Deadlocks

Introdujimos formas de coordinar *threads*, o de hacer que un *thread* espere al otro. Existen situaciones en que – por error – dos o más *threads* se esperan mutuamente, sin que ninguno finalmente avance. A este tipo de situaciones se le llama ***deadlock*** o **interbloqueo**, aunque hay nombres *menos afortunados*...

Abrazo mortal

Deadlock de tráfico

Veamos dos ejemplos concretos de ***deadlocks***.

Ejemplo con *locks*:

```
import threading
import time


lock_1 = threading.Lock()
lock_2 = threading.Lock()


def master() -> None:
    time.sleep(2)
    print("Master: adquiriendo lock_1")
    with lock_1:
        time.sleep(2)
        print("Master: adquiriendo lock_2")
        with lock_2:
            print("Master: ¡trabajando!")


def worker() -> None:
    time.sleep(1.5)
    print("Worker: adquiriendo lock_2")
    with lock_2:
        time.sleep(2)
        print("Worker: adquiriendo lock_1")
        with lock_1:
            print("Worker: ¡trabajando!")


t1 = threading.Thread(target=master)
t2 = threading.Thread(target=worker)

t1.start()
t2.start()
```

```
Worker: adquiriendo lock_2
Master: adquiriendo lock_1
Worker: adquiriendo lock_1
Master: adquiriendo lock_2
```

En el ejemplo anterior, tenemos dos *threads* y dos *locks*. El *thread master* alcanza a adquirir el `lock_1`, y el *thread worker* alcanza a adquirir el `lock_2`. Luego, *master* trata de adquirir `lock_2`, por lo que debe esperar que *worker* lo libere. Sin embargo, *worker* no liberará el `lock_2` sin antes poder adquirir `lock_1` . Es por esto que ni *master* ni *worker* nos dicen que están trabajando.

Ejemplo con *events*. Nota que es el mismo que el de la sección de señales entre *threads*, solo que cambiamos el orden en que se revisan/levantan las señales:

```
# Ejemplo adaptado de http://zulko.github.io/blog/2013/09/19/a-basic-example-of-threads-synchronization-in-python/

import threading
import time


# Tenemos dos eventos o señales.
# Esta es para avisar que el video ya está listo para ser reproducido.
video_cargado = threading.Event()

# Esta es para avisar que el audio ya está listo para ser reproducido.
audio_cargado = threading.Event()


def hora_actual():
    return time.ctime().split(" ")[3]


def reproducir_video(nombre: str) -> None:
    print(f"[{hora_actual()}] Cargando video {nombre}. Se demorará 10 segundos\n")
    time.sleep(10)
    print(f"[{hora_actual()}] Video cargado. Esperando audio")
    audio_cargado.wait()  # Esperar al audio
    video_cargado.set()  # Avisar que el video está listo

    print(f"[{hora_actual()}] Comenzando reproducción del video")


def reproducir_audio(nombre: str) -> None:
    print(f"[{hora_actual()}] Cargando audio {nombre}. Se demorará 3 segundos\n")
    time.sleep(3)
    print(f"[{hora_actual()}] Audio cargado. Esperando video")
    video_cargado.wait()  # Esperar al video
    audio_cargado.set()  # Avisar que el audio está listo

    print(f"[{hora_actual()}] Comenzando reproducción del audio")


t1 = threading.Thread(target=reproducir_audio, args=("Your Name - Zen Zen Zense",))
t2 = threading.Thread(target=reproducir_video, args=("Your Name - Zen Zen Zense",))

t1.start()
t2.start()
```

```
#
# Si descomentan las siguientes 2 líneas, la celda nunca se terminará
de ejecutar.
# Puesto que esperará que t1 termine. Situación que no ocurrirá.
#

# t1.join()
# t2.join()
```

```
[12:00:15] Cargando audio Your Name - Zen Zen Zense. Se demorará 3
segundos

[12:00:15] Cargando video Your Name - Zen Zen Zense. Se demorará 10
segundos

[12:00:18] Audio cargado. Esperando video
[12:00:25] Video cargado. Esperando audio
```

Esencialmente, en el ejemplo anterior estamos esperando que el otro *thread* avise que hizo su trabajo, antes de avisar que el *thread* actual hizo el suyo. Esto produce que `reproducir_video` espere a que la señal de `audio_cargado` se active, antes de activar `video_cargado`. Sin embargo, `reproducir_audio` está esperando que `video_cargado` se active para luego activar `audio_cargado` []. La que puede, puede, pero estos *threads* no pueden.

Lo importante – para ti como programador o programadora – es saber que los ***deadlocks*** pueden ocurrir, y que debes tener cuidado al programar para que esto no te pase.